

Multi-agent systems - Homework 6

Matuš Halák (2724858)

December 2025

Relevant code snippets are provided in the Code Appendix A.

1 Monte Carlo Tree Search

1.1 Question 1: Implementing MCTS for binary tree search.

To search for the target leaf node in a binary tree of depth 20 ($D_{MAX} = 20$) using Monte Carlo Tree Search (MCTS), we defined the leaf rewards as function of the edit distance $d(\cdot, \cdot)$ between the leaf addresses A_i and the target leaf address A_T .

$$r_i = B \exp(-d(A_i, A_T)/\tau) + \epsilon_i$$

where $B = 10$, $\tau = D_{MAX}/5$, and $\epsilon_i \sim N(0, 1)$.

We used the binary representation with 0 (left) and 1 (right). The randomly chosen target state had the address

$$A_T = 00000011000010010111$$

associated with the reward

$$r_T = 8.32$$

We applied MCTS with 50 iterations per "move" from each root node and used 3 random rollouts up to leaf nodes each time when estimating the value of previously unvisited nodes on the edge of the "snowcap". To balance exploration and exploitation, during tree search we used the UCB rule to decide actions within the "snowcap", with the hyperparameter $c = 5$.

$$\pi_{MCTS}(s) = \arg \max_a UCB(s, a)$$

$$UCB(s, a) = Q(s, a) + c \sqrt{\frac{\log N_s}{N_{sa}}}$$

where N_s represents the number of visits to the current state, and N_{sa} represents the number of visits to the child reached by taking action a in state s (in other words, the number of times the state-action pair (s, a) was encountered).

The Q -values of state-action pairs were updated based on estimates from immediate leaf rewards $\hat{r}(s, a)$ obtained either directly or through random rollouts up to the leaf. The Q -value backpropagating update was implemented as an incremental average.

$$Q^{t+1}(s, a) = Q^t(s, a) + \frac{\hat{r}(s, a) - Q^t(s, a)}{N_{sa}}$$

After each 50 iterations from a root node, the next action (0/1) was chosen greedily based on the latest Q -values, and the successor state was assigned as the new root node for the next round of MCTS for the next move.

$$\pi_{greedy}(s) = \arg \max_a Q^{t=50}(s, a)$$

To increase search efficiency and because of the naturally hashable state representation, we re-used previously encountered states and their Q -values and visit counts using a hash-map storing the node "addresses" as keys, and the Node objects as values.

Figure 1 shows the result of a single MC binary tree search run. We note that the target leaf node with the optimal reward is reached while only visiting 0.03 % of tree nodes.

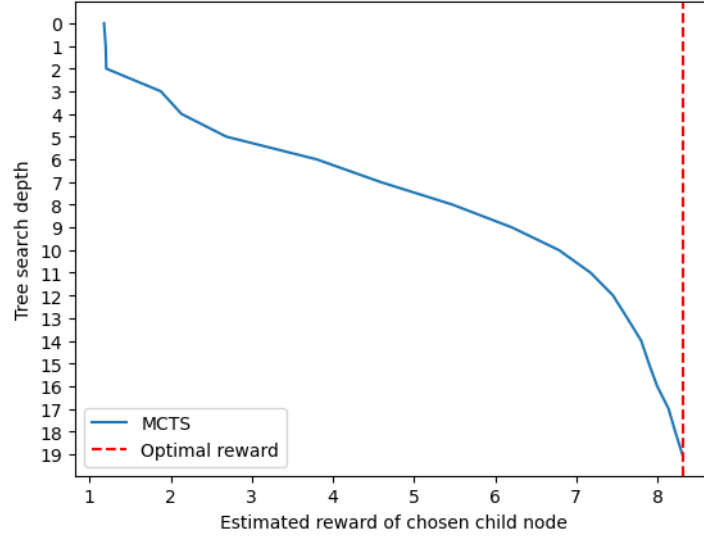


Figure 1: **MCTS for target leaf node.** 50 MCTS iterations per root, 3 rollouts per snowcap node, $c = 5$.

1.2 Question 2: The influence of UCB hyperparameter c .

Figure 2 shows the results of experimenting with the c hyperparameter on the performance of MCTS for binary tree search.

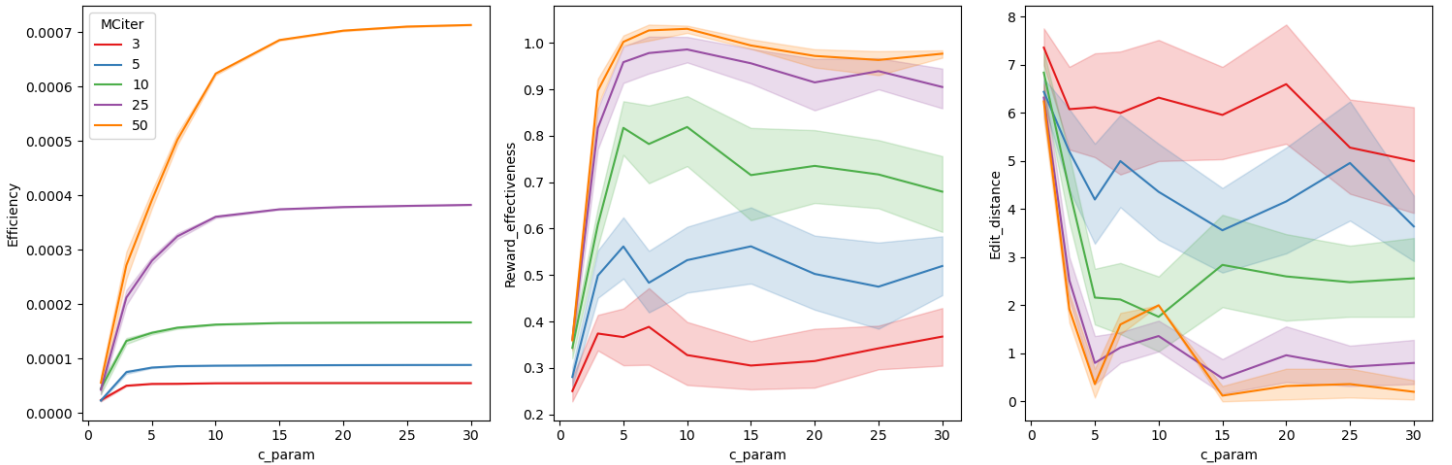


Figure 2: **Efficiency and effectiveness of MCTS as function of c hyperparameter.** *Left:* Efficiency (fraction of tree nodes visited). *Middle:* Effectiveness (fraction of optimal reward achieved). *Right:* Edit distance (of reached leaf node from target leaf node). Colors indicate different number of MCTS iterations from each root node. Shaded regions indicate 95% confidence intervals over 25 restarts with the same hyperparameters.

We see that influence of c on tree search efficiency is logarithmic. While the fraction of visited tree nodes initially increases rapidly with increasing c , this levels off and beyond certain c the efficiency remains constant regardless of increasing c . Using more MCTS iterations per root node increases the fraction of tree nodes visited, and shifts the slope of the c -efficiency curve.

The influence of c on reward effectiveness reflects the balance between exploration and exploitation. Low values of c ($c < 5$) encourage near pure exploitation of current Q-value estimates, which is especially problematic when using few MCTS iterations per root node. In the range $5 \leq c \leq 10$, the algorithm seems to successfully balance exploitation of Q-value estimates and exploration of under-visited states, and this is the region with the highest reward effectiveness (especially effective when using more than 10 MCTS iterations per root node). Finally, high values of c ($c > 10$) encourage too much exploration without making use of previous experience, leading to diminishing returns.

Finally, we note that greater exploration (marked by higher c), generally led to leaf nodes with lower edit distance to the target nodes, although the gains in target address similarity showed diminishing returns when increasing c beyond 5.

2 Markov Decision Process

Throughout this section, we denote the terminal state 12 as state 0. Clockwise action R is considered action 0 and counter-clockwise action L is considered action 1.

2.1 Question 1: Transition matrices and expected immediate rewards

We construct transition matrices for the MDP with $\epsilon = 0.45$ and show them in Figure 3.

$$T_R = p(s' | s, a = \text{clockwise}), \quad T_L = p(s' | s, a = \text{counter-clockwise})$$

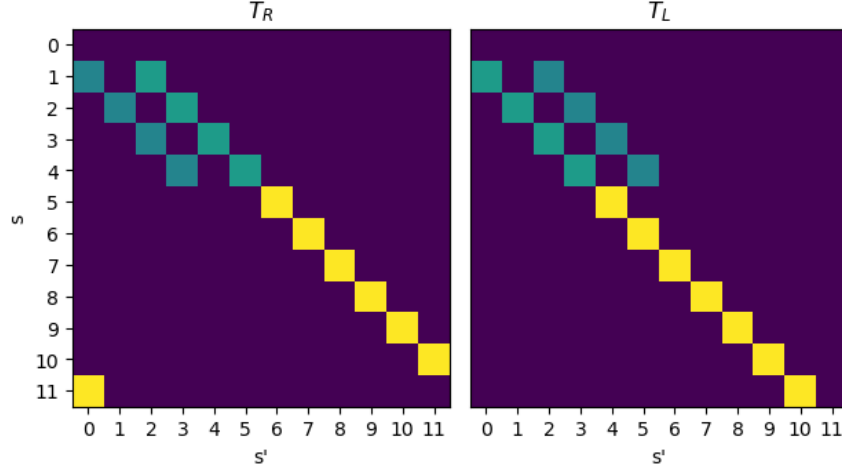


Figure 3: Transition probability matrices T_R and T_L for MDP with $\epsilon = 0.45$.

Explicitly for $\epsilon = 0.45$,

$$T_R = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0.45 & 0 & 0.55 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.45 & 0 & 0.55 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.45 & 0 & 0.55 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.45 & 0 & 0.55 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad T_L = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0.55 & 0 & 0.45 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.55 & 0 & 0.45 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.55 & 0 & 0.45 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.55 & 0 & 0.45 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Furthermore, we calculate the expected reward for each state action pair, taking into account the transition probabilities and immediate rewards.

$$\mathbb{E}[r(s, a)] = R_{s,a} = \sum_{s'} T_a(s, s') r(s, a, s')$$

Again, when taking the MDP with $\epsilon = 0.45$, we obtain the following states $\times$ actions expected reward matrix.

$$R = \begin{bmatrix} 0 & 0 \\ -0.55 & 5.5 \\ -0.55 & -0.55 \\ -0.55 & -0.55 \\ -0.55 & -0.55 \\ -1 & -1 \\ -1 & -1 \\ -1 & -1 \\ -1 & -1 \\ -1 & -1 \\ -1 & -1 \\ -1 & -1 \\ 10 & -1 \end{bmatrix}$$

2.2 Question 2: Optimal policy when $\epsilon = 0$

For any given ϵ we can solve for the optimal policy π^* by first performing value iteration to obtain the optimal values for each state $\mathbf{v}^*$, and subsequently in each state choosing actions that lead to highest combination of expected immediate rewards with successor state values. Namely for some preset convergence threshold Δ ,

$$\mathbf{v}^* = \mathbf{v}^t \quad \text{if} \quad \max_s \{\mathbf{v}^t - \mathbf{v}^{t-1}\} < \Delta$$

$$\mathbf{v}^t = \max_a \{\mathbf{R}_{:,a} + T_a \mathbf{v}^{t-1}\}$$

$$\pi^*(a|s) = \begin{cases} 1 & \text{if } a = \arg \max_{a'} (\mathbf{R}_{:,a'} + T_{a'} \mathbf{v}^*)_s \\ 0 & \text{otherwise} \end{cases}$$

Using the above-described value-iteration procedure, derived directly from $\mathbf{v}^* = \max_a (\mathbf{r}_a + T_a \mathbf{v}^*)$, for a fully deterministic MDP with $\epsilon = 0$, we obtain

$$\mathbf{v}_{\epsilon=0}^* = [0 \quad 10 \quad 9 \quad 8 \quad 7 \quad 6 \quad 5 \quad 6 \quad 7 \quad 8 \quad 9 \quad 10]$$

$$\pi_{\epsilon=0}^* = \begin{bmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0.5 & 0.5 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \end{bmatrix}$$

We note that both successor states of state 6 have the same value and immediate reward, $\mathbf{v}_{\epsilon=0}^*(6 \pm 1) = 6$. Therefore, to indicate indifference between choice of action in state 6 since both lead to identical returns, our implementation sets $\pi_{\epsilon=0}^*(a|s = 6) = 1/2 \quad \forall a$. Furthermore, we see that value iteration recovered the "common sense" policy of taking the shortest path to the absorbing state $s = 0$, by setting $\pi_{\epsilon=0}^*(a = L|s) = 1 \quad \forall s < 6$ and $\pi_{\epsilon=0}^*(a = R|s) = 1 \quad \forall s > 6$.

2.3 Question 3: Optimal policy when $\epsilon = 0.2$ or $\epsilon = 0.45$

Using the same value-iteration procedure we obtain the optimal value function and policy for stochastic versions of the MDP with $\epsilon = 0.2$ and $\epsilon = 0.45$.

$$\mathbf{v}_{\epsilon=0.2}^* = [0 \quad 9.6680 \quad 8.3398 \quad 7.0273 \quad 5.7773 \quad 4.7773 \quad 5 \quad 6 \quad 7 \quad 8 \quad 9 \quad 10]$$

$$\mathbf{v}_{\epsilon=0.45}^* = [0 \quad 8.1372 \quad 5.8605 \quad 4.3000 \quad 3.6150 \quad 4 \quad 5 \quad 6 \quad 7 \quad 8 \quad 9 \quad 10]$$

$$\pi_{\epsilon=0.2}^* = \begin{bmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \end{bmatrix} \quad \pi_{\epsilon=0.45}^* = \begin{bmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 1 & 0 \end{bmatrix}$$

The effect of higher ϵ is that choosing actions leading to the stochastic states 1–4 becomes more risky and unreliable, since higher epsilon means higher risk of transitioning to the unintended state and accumulating the -1 non-terminal transition

cost. This most directly affects policy in deterministic states adjacent to the stochastic states and stochastic states adjacent to deterministic states.

At $\epsilon = 0.2$ the chance of transitioning to the unintended non-terminal state is low, but even this low probability of accumulating additional non-terminal costs in the stochastic states is enough to break the tie in state 6 and change the policy to always take the equally-long but deterministically certain clockwise route from state 6 ($\pi_{\epsilon=0.2}^*(a = R|s = 6) = 1$). This is because $q_{\epsilon=0.2}^*(6, R) = 5 > q_{\epsilon=0.2}^*(6, L) = 3.7773$.

At $\epsilon = 0.45$, the chance of transitioning to the unintended state is almost equal to the chance of transitioning to the intended state. The consequence is that the deterministic state $s = 5$ which is closer to the terminal state via the stochastic counterclockwise route, switches to taking the longer deterministic clockwise route ($\pi_{\epsilon=0.45}^*(a = R|s = 5) = 1$), because it guarantees higher returns $q_{\epsilon=0.45}^*(5, R) = 4$ than the "shorter" but risky and stochastic counterclockwise route $q_{\epsilon=0.45}^*(5, L) = 2.615$.

We ran an additional experiment and saw that when the transitions in the stochastic states were practically random ($\epsilon = 0.49$), even the last stochastic state $s = 4$ switches to the clockwise route ($\pi_{\epsilon=0.49}^*(a = R|s = 4) = 1$), because the additional guaranteed cost is less than the expected cost in the stochastic direction.

2.4 Question 4: Parametrized policy

Figure 4 shows the results of our experiments with fitting a sigmoid-parametrized policy with Monte Carlo on-policy rollouts.

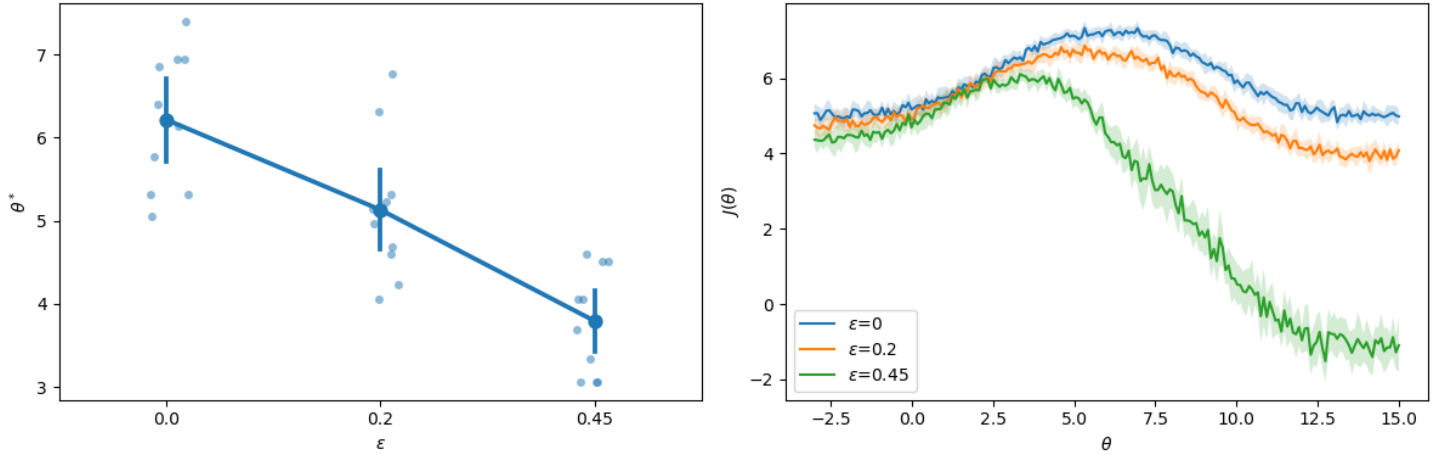


Figure 4: **Monte-Carlo fitting of parametrized policy, as function of MDP stochasticity.** *Left:* θ^* as function of ϵ . Error-bars reflect 95 % confidence intervals in θ^* estimates across re-runs, points indicate θ^* estimates from re-runs at the same ϵ , bold points indicate average θ^* across re-run estimates. *Right:* $J(\theta)$ as function of θ . Average and standard deviation across re-runs is shown. Data was collected over 10 re-runs with 200 on-policy rollouts for each (θ, ϵ) combination.

Because the policy parametrizes the probability of taking the clockwise route R in a given state s :

$$\pi_{\theta}(a = R|s) = \sigma(s - \theta)$$

θ^* can be seen as the optimal "pivot" state beyond which the taking the clockwise route should be preferred with higher probability.

Matching our analysis in the previous question, we see that at the stochasticity of the MDP increases, this pivot state θ^* shifts from just above 6 (since at $\epsilon = 0$ the two directions are tied) to 5 and 4 respectively, just like with value iteration.

Matching this observation are the curves of the expected total returns of under different (θ, ϵ) combinations. As stochasticity (ϵ) of the MDP increases, the peak of the curves shifts to the left, indicating a shift in the pivot state.

A Code Appendix

A.1 MCTS

```
class Node:
    # Class-level constants
    # max tree depth
    DMAX = 20
    # adress of target node
    seed = np.random.seed(2025)
    At = np.random.randint(0,2, DMAX)
    # Leaf reward params
    B = 10
    tau = DMAX/5
    # number of random rollouts to estimate node values
    nrollout = 3

    def __init__(self,
                  action = None,
                  parent = None):
        self.state_visits = 1.0
        self.root = (parent is None) and (action is None)
        self.parent:Node = parent
        if self.root: # root node
            self.adress = None
            self.leaf = False
        else: # intermediate or leaf node
            if parent.root:
                self.adress = np.array([action])
            else:
                self.adress = np.append(parent.adress, action)
            self.leaf = len(self.adress) == self.DMAX

        if not self.leaf: # not leaf node
            self.actions = [0,1]
            self.children = {a: None for a in self.actions}
            self.child_visits = {a:1.0 for a in self.actions}
            self.Qs = {a:0 for a in self.actions}
        else: # leaf node
            self.actions = None
            self.children = None
            self.reward = leaf_reward(self.adress, self.At,
                                      self.B, self.tau)

    def add_child(self, action:int):
        assert not self.leaf, 'Leaf nodes cannot add children'
        ChildNode = Node(action=action, parent=self)
        self.children[action] = ChildNode
        return ChildNode

    def rollout(self):
        """
        Performs random rollout(s) to estimate node value
        """
        fresh_rng = np.random.default_rng(None)
        steps_to_go = self.DMAX - len(self)
        # Estimate node value with rollout(s)
        rollout_actions = fresh_rng.integers(0,2, (self.nrollout, steps_to_go))
```

```
leaf_val = 0
for irollout in range(self.nrollout):
    leaf = np.append(self.adress, rollout_actions[irollout,:])
    leaf_val += leaf_reward(leaf, self.At, self.B, self.tau)
node_est = leaf_val / self.nrollout
return node_est
```

```

def MCTS(Root:Node, StateMap:defaultdict,
        MCTS_iter:int, c:float):
    '''
    Monte Carlo Tree Search Algorithm with UCB rule
    '''
    # Restart MCTS from root node for fixed number of iterations
    for it in range(MCTS_iter):
        State:Node = Root
        path:list[Node] = [State]

        # Snowcap according to UCB
        while not State.leaf:
            # Compute UCB for different possible actions
            UCBs = [State.Qs[a] + c * np.sqrt(
                np.log(State.state_visits) / State.child_visits[a])
                    for a in State.actions]
            # Choose action according to UCB rule
            best = State.actions[np.argmax(UCBs)]
            if State.children[best] is None:
                State = State.add_child(best)
            else:
                State = State.children[best]
            # path in current iteration
            path.append(State)
            # keep hash map of previously encountered states
            if StateMap[str(State)] is None:
                StateMap[str(State)] = State
            break

        # Immediate leaf reward / Rollout at edge of snowcap
        R = State.reward if State.leaf else State.rollout()

        # Backpropagate reward up through snowcap
        for Step in path[::-1]:
            Step.state_visits += 1

            # up to root node
            if Step.parent is not None:
                # Add visits to actions that created you
                parent_move = Step.adress[-1]
                Step.parent.child_visits[parent_move] += 1
                # Incremental average on Q-values
                Step.parent.Qs[parent_move] += (R - Step.parent.Qs[parent_move]
                                                  ) / Step.parent.child_visits[parent_move]

    if Root.leaf:
        return Root.reward, Root, StateMap
    else:
        # After MCTS choose action greedily based on Q-values
        # (UCB already balances exploration and exploitation during MCTS)
        best_action = int(np.argmax([Root.Qs[a] for a in Root.actions]))
        return Root.Qs[best_action], Root.children[best_action], StateMap

```

```

def MCTS_binary_tree_search(MCTS_iter:int = 50, c:float = 5, PRINT:bool = False,
                             path:bool = False):
    State = Node()
    StateMap = defaultdict(lambda: None)
    if path: pth = []
    while not State.leaf:
        est_r, State, StateMap = MCTS(Root=State, StateMap=StateMap,
                                       MCTS_iter=MCTS_iter, c=c)
        if path: pth.append(est_r)

    optimal_reward = leaf_reward(State.At, State.At, State.B, State.tau)
    total_nodes = sum(2**d for d in range(State.DMAX))
    log = dict(
        MCiter = MCTS_iter,
        c_param = c,
        optimal_reward = optimal_reward,
        final_reward = est_r,
        Efficiency = len(StateMap) / total_nodes,
        Reward_effectiveness = est_r/optimal_reward,
        Edit_distance = edit_dist(State.adress, State.At)
    )
    ... # printing and plotting in notebook
    return str(State), log

def MCTSexperiments(MCgrid = [3, 5, 10, 25, 50],
                    cgrid = [1, 3, 5, 7, 10, 15, 20, 25, 30],
                    restarts_per_setup:int = 10):
    results = []
    # Collect results
    for mc in MCgrid:
        for cp in cgrid:
            for _ in range(restarts_per_setup):
                _, res = MCTS_binary_tree_search(mc, cp)
                results.append(res)

    # Organize results
    ResDF = DataFrame(results)

    # Plot results
    ... # included in notebook

```

A.2 MDP

```
def makeTransProb(eps:float):  
    '''  
    Rows FROM, Cols TO  
    '''  
  
    assert 0 <= eps < 0.5  
    # Right: clockwise  
    Ropp = np.diag(np.full(4, eps), -1)  
    Tright = np.eye(N = 12, k = 1)  
    Tright[0,:] = 0  
    Tright[:5, :] *= (1-eps)  
    Tright[:5, :5] += Ropp  
    Tright[-1, 0] = 1  
    # Left: counterclockwise  
    Lopp = np.diag(np.full(4, eps), 1)  
    Tleft = np.eye(N = 12, k = -1)  
    Tleft[:5, :] *= (1-eps)  
    Tleft[1:6, 1:6] += Lopp  
  
    return Tright, Tleft  
  
def makeImmediateReward(NT:float = -1, T:float = 10):  
    # 0: right clockwise  
    rR = np.eye(12, k=1) * NT  
    rR[0,:] = 0  
    rR[-1, 0] = T # 11->12 transition  
    # 1: left counterclockwise  
    rL = np.eye(12, k=-1) * NT  
    rL[1, 0] = T # 12<-1 transition  
  
    return rR, rL  
  
def makeExpectedReward(T:tuple[np.ndarray, np.ndarray],  
                        r:tuple[np.ndarray, np.ndarray]  
                        )-> np.ndarray:  
  
    TR, TL = T  
    RR, RL = r  
    ErR = (TR * RR).sum(axis = 1)  
    ErL = (TL * RL).sum(axis = 1)  
    Er = np.column_stack((ErR, ErL))  
  
    return Er
```

```

def find_policy(eps:float, delta:float = 1e-5,
               NT:float = -1, T:float = 10,
               PRINT:bool = True):
    TR, TL = makeTransProb(eps)
    TProb = np.dstack((TR, TL)).transpose((2, 0, 1))
    ER = makeExpectedReward((TR, TL), makeImmediateReward(NT, T))
    vstar = value_iteration(ER, TProb, delta)
    policy = policy_from_value(vstar, ER)
    if PRINT:
        print(f'Value iteration for MDP with epsilon = {eps}')
        print(f'V Star:\n {vstar}')
        print(f' Star:\n {policy}')
    return vstar, policy

def value_iteration(ER:np.ndarray, TProb:np.ndarray,
                   delta:float = 1e-5,
                   )->np.ndarray:
    # Start value iteration with expected reward
    v0 = ER.mean(axis = 1)
    delta_t = 1e3
    # Value iteration until small enough
    while delta_t >= delta:
        v1 = np.max(ER + (TProb @ v0).T,
                    axis = 1)
        delta_t = max(v1 - v0)
        v0 = v1
    return v1

def policy_from_value(vstar:np.ndarray, er:np.ndarray)->np.ndarray:
    policy = np.zeros((len(vstar)-1, 2))
    values = np.zeros_like(policy)

    # Clockwise move
    values[:,0] = np.roll(vstar, -1)[1:]
    # Counterclockwise move
    values[:,1] = np.roll(vstar, 1)[1:]
    # Add immediate rewards
    values += er[1:, :]
    # Best moves
    best = np.argmax(values, axis = 1)
    # Ties
    ties = (values[:, 0] == values[:, 1])
    # Policy
    policy[np.arange(len(best)), best] = 1
    policy[ties,:] = 1/2
    return policy

```

```

def param_policy(state:int, theta:float)->float:
    '''
    Policy parametrizing  $p(a = R \mid s = state)$ 
    '''
    sigmoid = lambda x: 1 / (1 + np.exp(-x))
    return sigmoid(state-theta)

def policy_rollout(theta:float, eps:float,
                    nt:float = -1, t:float = 10
                    )->float:
    # transition probabilities
    Transition:tuple[np.ndarray] = makeTransProb(eps)
    # expected immediate rewards
    Reward:tuple[np.ndarray] = makeImmediateReward(nt, t)
    starting_states = np.arange(1, 12)
    successor_states = np.arange(12) # incl. terminal
    # action probabilities (in each state)
    pR = param_policy(starting_states, theta)
    pL = 1 - pR
    prob_a = np.column_stack((pR, pL))
    # rollout
    state = np.random.choice(starting_states)
    cumulative_reward = 0
    while state != 0:
        # choose action according to policy
        action = np.random.choice([0,1], p = prob_a[state-1])
        T = Transition[action]
        R = Reward[action]
        # successor according to transition probability
        successor = np.random.choice(successor_states, p = T[state])
        # assign immediate reward
        cumulative_reward += R[state, successor]
        # continue rollout
        state = successor
    return cumulative_reward

def fit_param_policy_MC(theta_grid:np.ndarray, eps:float,
                        nrollouts:int = 10,
                        nt:float = -1, t:float = 10):
    theta_rewards = np.zeros_like(theta_grid)

    # Obtain MC estimates of the various theta params
    for itheta, theta in enumerate(theta_grid):
        for _ in range(nrollouts):
            theta_rewards[itheta] += policy_rollout(theta, eps, nt, t)

    # Choose theta param with the highest expected return
    theta_rewards /= nrollouts
    theta_star_idx = np.argmax(theta_rewards)
    theta_star = theta_grid[theta_star_idx]

    return theta_star, theta_rewards

```

```

def MDPexperiments(theta_grid: np.ndarray, eps_grid: np.ndarray,
                    nrollouts: int, reruns: int,
                    nt: float = -1, t: float = 10,
                    n_jobs: int = -1):

    rows = []
    curves = {} # eps -> (mean_curve, std_curve)

    # Collect data from different runs
    for eps in eps_grid:
        res = Parallel(n_jobs=n_jobs)(
            delayed(fit_param_policy_MC)(theta_grid, eps, nrollouts, nt, t)
            for _ in range(reruns)
        )

        thetas = np.array([r[0] for r in res], dtype=float)
        Jcurves = np.stack([r[1] for r in res], axis=0) # (reruns, |theta_grid|)

        # store per-run summary
        for i, th in enumerate(thetas):
            rows.append({"eps": float(eps), "theta_star": float(th), "J_star": float(max(Jcurves[i,:]))})

        # store mean/std curves for plotting
        curves[float(eps)] = (Jcurves.mean(axis=0), Jcurves.std(axis=0))

    df = DataFrame(rows)
    ...# plotting in notebook

```